

Lecture 22: More Kokkos

Anders Johansson
ajohans@sandia.gov

CS205 4/16/2026
Notebook: bit.ly/cs205lec22



Harvard John A. Paulson
School of Engineering
and Applied Sciences

What is Kokkos?

A library that provides what we need to write portable parallel code

Data structures and parallel launch policies, KokkosKernels = BLAS etc.

At compile time, it translates the expressed parallelism to
CUDA/OpenMP/HIP/SYCL/etc.

```
cmake .. -DKokkos_ENABLE_CUDA=ON (or -DKokkos_ENABLE_OPENMP=ON)
```

Multidimensional loops

Replace the implicit `RangePolicy` with a `MDRangePolicy<Rank<n>>`

```
Kokkos::parallel_for("name",  
    Kokkos::MDRangePolicy<Kokkos::Rank<3>>({0,0,0}, {n1,n2,n3}),  
    KOKKOS_LAMBDA (int i, int j, int k) {...}  
);
```

Hierarchical parallelism

`RangePolicy` and `MDRangePolicy` hide the details of thread blocks and warps

Sometimes, being aware of them can allow for optimizations

Examples: Use of shared memory, per-warp or per-block reductions

Kokkos terminology:

Kokkos	OpenMP	CUDA
Team	Thread	Thread block
Thread	-/thread	Warp
Vector	SIMD	Thread

Parallelization levels:

- `TeamThreadRange`: over warps
- `ThreadVectorRange`: over threads within warp
- `TeamVectorRange`: over warps and threads

Hierarchical parallelism: syntax

```
Kokkos::parallel_for("test",  
    Kokkos::TeamPolicy<>(N, nthreads, nvector),  
    KOKKOS_LAMBDA(Kokkos::TeamPolicy<>::member_type team) {  
        int teamid = team.league_rank(); // normally only this on  
        int nteams = team.league_size();  
        int teamsize = team.team_size();  
        int threadid = team.team_rank(); // "thread" ~= "warp"  
  
        ...  
    });
```

`nvector` is either an integer or omitted (1)

`nthreads` is an integer or `Kokkos::AUTO()`

Hierarchical thread mapping

Within a parallel for over a `TeamPolicy`, launch parallel loops over inner ranges:

```
Kokkos::parallel_for("test",  
    Kokkos::TeamPolicy<>(N, nthreads, nvector),  
    KOKKOS_LAMBDA(Kokkos::TeamPolicy<>::member_type team) {  
        int i = team.league_rank();  
  
        Kokkos::parallel_for(  
            Kokkos::TeamVectorRange(team, n),  
            [&] (int j) {  
                ...  
            }  
        );  
    });
```

Hierarchical thread mapping

Within a parallel for over a `TeamPolicy`, launch parallel loops over inner ranges:

```
Kokkos::parallel_for("test",
    Kokkos::TeamPolicy<>(N, nthreads, nvector),
    KOKKOS_LAMBDA(Kokkos::TeamPolicy<>::member_type team) {
        int i = team.league_rank();
        Kokkos::parallel_for(
            Kokkos::TeamThreadRange(team, n),
            [&] (int j) {
                Kokkos::parallel_for(
                    Kokkos::ThreadVectorRange(team, n),
                    [&] (int k) {...}
                );
            }
        );
    }
}
```

Do operation only once per team:

```
Kokkos::single(Kokkos::PerTeam(team), [&] () {...});
```

Team-wide barrier:

```
team.team_barrier();
```

Examples

See colab

Shared memory

CUDA and Kokkos offer per-block/per-warp shared (“scratch”) memory

Ask for shared memory when creating the policy:

```
Kokkos::TeamPolicy<>(...).set_scratch_size(  
    0, Kokkos::PerTeam(size)  
)
```

Level 0 = fastest, level 1 = usually normal memory

Can be either per team or per thread (or both)

See <https://kokkos.org/kokkos-core-wiki/API/core/policies/TeamPolicy.html>

Getting shared memory

Inside the kernel:

```
ScratchVec x(team.team_scratch(0), N);
```

Views in the shared memory:

```
typedef Kokkos::View<  
    float*,  
    typename Kokkos::DefaultExecutionSpace::scratch_memory_space  
> ScratchVec;
```

This can be used to calculate required size:

```
int shmem_size = ScratchVec::shmem_size(N);
```

Summary

Simple 1D and multi-D loops in Kokkos are easy

Hierarchical parallelism offers more control

Shared memory can help if a team reads the same memory multiple times

Further reading: <https://kokkos.org/kokkos-core-wiki/>

- Programming guide, chapters 1–8
- Lectures 1–4.2